

Proyecto 1

Whack-a-mole: juego híbrido FPGA / lógica discreta

Profesor

Dr.-Ing. Jeferson González-Gómez
Ing. Rolén Coto Calderón

1. Introducción

Este proyecto le permitirá al estudiante integrar sus conocimientos sobre sistemas combinacionales, secuenciales, *datapath* y *control path*, para proponer una solución a un problema de diseño y llevarla a una implementación en FPGA mediante el uso de lenguajes de descripción de hardware. En este proyecto, la solución completa no reside únicamente en la FPGA: parte del sistema deberá implementarse con lógica discreta (integrados de la familia 74xx) montada en *proto-board*, y ambas partes deberán comunicarse entre sí mediante un enlace serial.

En este primer proyecto, los estudiantes implementarán una versión del juego electrónico *Whack-a-mole* (“golpea al topo”). El juego consiste tradicionalmente en un tablero con varios agujeros de los que emergen “topos” de forma aleatoria; el jugador debe golpear al topo correspondiente antes de que se esconda nuevamente, acumulando puntos por cada acierto.

El juego se desarrolla, para efectos de este proyecto, de la siguiente manera:

- Un circuito montado en *proto-board*, construido únicamente con lógica discreta, decide de forma pseudoaleatoria cuál de 8 posiciones posibles es la del “topo” que aparece en cada turno, y lo indica de forma visible mediante un LED.
- El circuito discreto envía dicha posición a la FPGA mediante un enlace serial.
- Toda la lógica de control del juego — turnos, tiempos, conteo de aciertos y fallos, dificultad progresiva y despliegue de resultados — se implementa en la FPGA.
- El jugador “golpea” al topo presionando el *push button* externo asociado a la posición indicada, conectado a la FPGA mediante GPIO, antes de que expire la ventana de tiempo del turno.

La motivación de combinar ambas tecnologías es que los estudiantes experimenten, en un solo proyecto, tanto el diseño de sistemas digitales síncronos descritos en HDL y sintetizados en FPGA, como las limitaciones y particularidades del diseño con lógica discreta: generación de señales de reloj propias, ausencia de una referencia de tiempo compartida entre los dos subsistemas, y el reto de establecer una comunicación confiable entre dominios de reloj independientes.

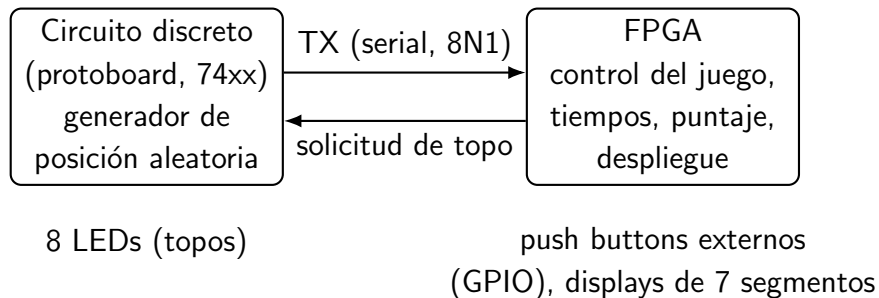


Figura 1: Arquitectura general del sistema: el subsistema discreto genera la posición del topo y la transmite por serial; la FPGA concentra toda la lógica de control del juego.

Indicaciones generales:

- Lea y trate de comprender todo el trabajo solicitado antes de iniciarlo.
- En la documentación de los módulos a diseñar en este proyecto, los estudiantes deben incluir las tablas de verdad, diagramas de bloque y, para el circuito discreto, diagramas esquemáticos propios correspondientes a cada diseño desarrollado.
- Para la presentación funcional se le pedirá que muestre los circuitos propuestos, tanto en protoboard como en la FPGA, junto con las simulaciones correspondientes de la parte descrita en HDL.
- Asegúrese de que el avance del proyecto quede protocolado con suficientes *commits* en el repositorio, tanto de la documentación como del código HDL.

El subsistema implementado en la FPGA debe ser completamente **sintetizable**, descrito utilizando diseño RTL con SystemVerilog. Tome en cuenta que estará desarrollando lógica tanto combinacional como secuencial, por lo que los bloques desarrollados deben estar libres de elementos de memoria no intencionados, especialmente *latches*. Compruebe, por medio de los mensajes, visualización del *netlist* de la herramienta, y la utilización del *linter*, que no se generen dichos elementos como parte del proceso de síntesis. Las simulaciones deben ser presentadas a nivel de **post-implementación** temporizadas. El circuito discreto, por su parte, debe documentarse mediante un diseño esquemático propio del grupo, tablas de verdad de cada bloque combinacional, y un diagrama de estados o de tiempos para cada bloque secuencial.

Para el desarrollo de este proyecto se deberán acatar las siguientes indicaciones:

1. Lea y trate de comprender todo el trabajo solicitado antes de iniciarlo.

2. Durante el desarrollo del diseño y la validación, asegúrese de documentar todas las tablas de verdad, circuitos, figuras, diagramas (flujo, estado, bloques, tiempos), entre otros, que sean necesarios. Esto será **fundamental** para la revisión del diseño y para la documentación del proyecto.
3. Para la presentación funcional, se le pedirá que muestre algunos de los diseños propuestos, ya sea en implementación o simulación, tanto del lado discreto como del lado FPGA. Dicha selección se hará el día de la presentación.
4. Todos los bloques de diseño desarrollados en la FPGA deben ser **sintetizables** en System-Verilog. El sistema debe funcionar con un único reloj de entrada 100 *Mhz* del lado de la FPGA.
5. El circuito discreto **no puede** compartir su reloj o señal de temporización con la FPGA: ambos subsistemas deben operar con referencias de tiempo independientes, y la comunicación entre ellos debe resolver dicha diferencia mediante el protocolo serial indicado.

2. Investigación previa

Para el desarrollo exitoso de este proyecto se debe investigar sobre los siguientes aspectos técnicos:

1. Qué es el modelado de comportamiento y de estructura en diseño digital.
2. El proceso de síntesis lógica en el diseño de circuitos digitales, tanto para el desarrollo de un ASIC como para una FPGA.
3. Tecnología de FPGAs. Estudie el funcionamiento de la lógica programable en general, así como los componentes básicos de un chip de FPGAs.
4. Investigue cuál es el funcionamiento de las máquinas de estado finitos. Explique la diferencia entre una máquina de Moore y una de Mealy, y muestre la diferencia por medio de diagramas de estados y señales.
5. Conceptos de *setup time* y *hold time*. ¿Qué importancia tienen en el diseño de sistemas digitales?
6. Conceptos de tiempos de propagación y tiempos de contaminación en circuitos combinacionales. Ruta crítica y cómo esta afecta en el diseño de sistemas digitales complejos. Investigue su relación con la frecuencia máxima de operación de un circuito.
7. Investigue sobre las mejores prácticas para la asignación de relojes y división de frecuencia en FPGAs. En este apartado haga énfasis en el uso de las entradas habilitadoras de reloj (*clock enables*) presentes en las celdas de la FPGA, para lograr tener tiempos de ejecución diferentes a lo largo del sistema mientras se utiliza un solo reloj.
8. Investigue sobre el fenómeno de *rebotes* y ruido en pulsadores e interruptores. Defina qué técnicas digitales y analógicas (circuitos) se utilizan para cancelar este fenómeno. Además, investigue sobre los problemas de metastabilidad cuando se tienen entradas asíncronas en

circuitos digitales, y cómo se resuelven mediante sincronizadores de dos etapas (*double flip-flop synchronizer*).

9. Investigue el funcionamiento de un registro de desplazamiento (*shift register*) implementado con integrados discretos (por ejemplo, familia 74xx), tanto en su modalidad serie-a-paralelo como paralelo-a-serie, y su uso para implementar comunicación serial en hardware discreto.
10. Investigue la teoría de generación de números pseudoaleatorios mediante registros de desplazamiento con retroalimentación lineal (*Linear Feedback Shift Register*, LFSR): polinomios de realimentación, taps, y período máximo de la secuencia generada. Analice las diferencias prácticas de implementar un LFSR con integrados discretos frente a describirlo en HDL.
11. Investigue el funcionamiento de un decodificador (por ejemplo, un 74LS138 o equivalente) y su uso para seleccionar una de N líneas de salida a partir de una palabra binaria.
12. Investigue el protocolo de comunicación serial asíncrona UART: formato de trama (bit de inicio, bits de datos, bit de paridad, bit(s) de parada), concepto de *baud rate*, y por qué dos dispositivos sin reloj compartido pueden comunicarse de forma confiable si ambos acuerdan de antemano dicha velocidad de transmisión.

3. Especificaciones

El sistema completo se divide en dos subsistemas que se comunican mediante un enlace serial: el subsistema discreto, montado en *protoboard* con integrados de la familia 74xx, y el subsistema de control, implementado en la tarjeta con FPGA. A continuación se detallan las especificaciones de cada uno, así como las del enlace de comunicación entre ambos.

3.1. Subsistema discreto (protoboard)

El subsistema discreto es responsable de decidir, de forma pseudoaleatoria, cuál de 8 posiciones posibles corresponde al “topo” en cada turno del juego, de indicarlo visualmente, y de transmitir dicha posición a la FPGA. Todo este subsistema debe construirse únicamente con integrados de lógica discreta (familia 74xx u equivalente) y componentes discretos pasivos; no se permite el uso de microcontroladores, FPGAs adicionales, ni ningún dispositivo programable.

- Diseñe un generador de números pseudoaleatorios de al menos 3 bits, implementado como un LFSR construido con flip-flops discretos (por ejemplo, 74LS74 o 74HC164) y una red de realimentación XOR propia de su diseño. El diseño de la red de realimentación (*taps*), así como el esquemático completo, debe ser propio de cada grupo; no se entregará un esquemático de referencia. Documente su elección del polinomio de realimentación y el período de la secuencia resultante.
- Utilice un decodificador de 3 a 8 líneas (por ejemplo, un 74LS138) para traducir el valor de 3 bits generado por el LFSR en la activación de una de 8 salidas. Conecte cada una de dichas salidas a un LED individual, de forma que en todo momento se ilumine exactamente un LED que representa la posición del topo activo.
- El subsistema discreto debe generar una nueva posición aleatoria cada vez que reciba, desde la FPGA, un pulso de solicitud (“*siguiente topo*”). Diseñe la lógica de control necesaria (por ejemplo, mediante un flip-flop o pequeña máquina de estados discreta) para que el LFSR avance una única vez por cada solicitud recibida, y no de forma continua.
- Una vez generada la nueva posición, el subsistema discreto debe empaquetarla en una trama serial de 8 bits (ver sección 3.2) y transmitirla a la FPGA. Diseñe, con lógica discreta, la generación de la señal de reloj de transmisión (*baud rate*) — por ejemplo, mediante un oscilador astable (555 u equivalente) seguido de un divisor de frecuencia — así como el registro de desplazamiento paralelo-a-serie (por ejemplo, un 74LS165) encargado de anteponer el bit de inicio, desplazar los 8 bits de datos, y anteponer el bit de parada.

3.2. Enlace de comunicación serial

La comunicación entre el subsistema discreto y la FPGA se realiza mediante UART asíncrono, con el siguiente formato de trama:

- 1 bit de inicio (*start bit*).

- 8 bits de datos, sin bit de paridad.
- 1 bit de parada (*stop bit*).

Cada grupo debe definir y documentar la velocidad de transmisión (*baud rate*) que utilizará, tomando en cuenta que dicha velocidad debe ser lo suficientemente baja como para poder generarse de forma confiable con el oscilador discreto del punto anterior, y lo suficientemente estable como para que la FPGA pueda muestrear correctamente cada bit. Justifique en su documentación la velocidad elegida y la tolerancia de error entre los dos relojes independientes (el del protoboard y el de la FPGA) que dicha velocidad puede admitir.

Del lado de la FPGA, el byte recibido debe interpretarse de manera que sus 3 bits menos significativos codifiquen la posición del topo (0 a 7); los bits restantes pueden usarse libremente o fijarse en cero, según el diseño del grupo, siempre que se documente dicha convención.

3.3. Subsistema de control (FPGA)

Toda la lógica de control del juego reside en la FPGA y debe describirse en SystemVerilog, sintetizable, operando con un único reloj de entrada de 100 MHz.

- Implemente un receptor UART (compatible con el formato de la sección 3.2) que reciba la trama enviada por el subsistema discreto. La señal de recepción proviene de un dominio de reloj independiente del de la FPGA: diseñe la sincronización de dicha entrada mediante un sincronizador de dos etapas antes de procesarla.
- Las entradas de “golpe” deben implementarse mediante 8 *push buttons* externos (uno por posición), montados fuera de la tarjeta y conectados a la FPGA a través de sus pines de GPIO; no se permite usar los botones o *switches* integrados en la tarjeta para esta función. Toda la lógica de control de estas entradas — filtrado de rebotes y sincronización de las pulsaciones — debe implementarse en la FPGA mediante los circuitos secuenciales correspondientes.
- Cada turno, la FPGA debe: (1) enviar el pulso de solicitud al subsistema discreto, (2) recibir la posición del nuevo topo, (3) abrir una ventana de tiempo durante la cual dicha posición se considera “activa” y puede ser golpeada, y (4) verificar si, dentro de esa ventana, el jugador presionó el botón correspondiente a la posición correcta.
- Un golpe se considera acierto si el botón presionado corresponde a la posición activa y ocurre dentro de la ventana de tiempo del turno; en cualquier otro caso (botón incorrecto, o ninguno presionado antes de que expire la ventana) se considera un fallo.
- **Duración del topo activo y dificultad progresiva.** La ventana de tiempo durante la cual el topo permanece activo inicia en 1,5 s. Cada acierto consecutivo reduce dicha ventana en 100 ms, hasta un mínimo de 500 ms; alcanzado ese mínimo, la ventana se mantiene constante mientras el jugador siga acertando. Un fallo no aumenta la ventana de vuelta a su valor inicial: la dificultad alcanzada se conserva mientras la partida continúe. Implemente esta reducción utilizando *clock enables* derivados del reloj principal, de forma que no se generen relojes derivados adicionales. Documente, mediante una tabla, la relación entre el número de aciertos consecutivos y la duración resultante de la ventana.

- **Visualización.** Utilice 4 displays de 7 segmentos: 2 dígitos para la cantidad de aciertos acumulados desde el último reinicio (00 a 99), y 2 dígitos para la cantidad de fallos acumulados en el mismo período (00 a 99). Utilice además al menos un LED de la tarjeta como indicador de estado del juego, distinguiendo entre partida activa y partida terminada.
- **Oportunidades de reintento (vidas).** El jugador dispone de un máximo de 3 fallos consecutivos por partida. Un acierto reinicia a cero el conteo de fallos consecutivos, sin afectar el contador acumulado de fallos que se muestra en los displays. Al alcanzar el tercer fallo consecutivo, la partida termina: el sistema debe permanecer al menos 2 s en un estado de fin de partida, claramente distinguible mediante el LED de estado indicado en el punto anterior, antes de reiniciarse automáticamente con una nueva secuencia, contadores en cero y la ventana de tiempo de vuelta a su valor inicial de 1,5 s.
- Utilice el botón central de la tarjeta para reiniciar el juego manualmente en cualquier momento: dicho reinicio debe restablecer los contadores de aciertos y fallos, la ventana de tiempo a su valor inicial, y solicitar una nueva posición al subsistema discreto.

4. Evaluación

Este proyecto tiene una duración de tres semanas a partir de su inicio. El planteamiento del diseño debe ser validado con el profesor **previo a la presentación funcional**. Ningún proyecto será evaluado sin el visto bueno desde la perspectiva de diseño.

Las fechas oficiales de este proyecto son:

- **Inicio:** G1: 04.08.2026 / G2: 03.08.2026
- **Planteamiento del diseño:** G1: 13.08.2026 / G2: 12.08.2026
- **Entrega del informe técnico:** G1: 25.08.2026 / G2: 24.08.2026
- **Fin:** G1: 25.08.2026 / G2: 24.08.2026

El presente proyecto tiene un valor del **15 %** de la nota final del curso y se evaluará mediante cuatro rubros, cada uno calificado con una rúbrica común a todos los proyectos del curso. El detalle de cada rúbrica se presenta en el Anexo A.

1. Diseño	20 %
2. Presentación funcional	60 %
3. Informe técnico	15 %
4. Repositorio	5 %

El formato de entrega de cada documento (Markdown enriquecido, ubicado en la carpeta correspondiente del repositorio) se detalla junto a su rúbrica en el Anexo A.

A. Anexo: Rúbricas de evaluación

Las siguientes rúbricas son comunes a todos los proyectos del curso y se detallan a continuación para que el estudiante cuente con los criterios de evaluación completos dentro de este mismo documento.

A.1. Diseño

Documentación del diseño

Rúbrica aplicable al planteamiento del diseño entregado en cada proyecto del curso. El diseño debe seguir la metodología de diseño modular vista en clase. Evalúa la calidad de los diagramas, la justificación técnica de las decisiones tomadas, la completitud del documento y la claridad general de la presentación.

Formato de entrega: el planteamiento del diseño debe entregarse como un archivo Markdown (.md) enriquecido, ubicado en la carpeta docs/diseño/ del repositorio del proyecto. No se aceptan formatos alternativos, como un PDF exportado desde un procesador de texto externo, como entrega oficial.

Nivel	Descripción del criterio
Diagramas y representaciones del diseño (peso: 30 %)	
Excelente	Diagramas claros, correctos y bien organizados (bloques, flujo, estados, esquemas u otros). Reflejan correctamente la arquitectura del sistema y facilitan la comprensión.
Satisfactorio	Diagramas mayormente correctos, pero con detalles poco claros, incompletos o escasa explicación.
Regular	Diagramas simplificados o con errores que dificultan entender el diseño.
Deficiente	No hay diagramas o son incorrectos/confusos.
Justificación y decisiones de diseño (peso: 30 %)	
Excelente	Explica y justifica claramente las decisiones de diseño (arquitectura, señales, ecuaciones, implementación, etc.). Demuestra razonamiento técnico sólido.
Satisfactorio	Presenta las decisiones principales, pero con justificación limitada.
Regular	Justificación mínima o superficial de las decisiones de diseño.
Deficiente	No se explican ni justifican las decisiones de diseño.
Completitud del documento de diseño (peso: 20 %)	
Excelente	Incluye todos los elementos solicitados (objetivos, tablas, ecuaciones, diagramas, estrategia de implementación, plan de pruebas, etc.) de forma completa y bien estructurada.

Nivel	Descripción del criterio
Satisfactorio	Incluye la mayoría de los elementos requeridos, con pequeños faltantes o secciones poco desarrolladas.
Regular	Varias secciones importantes faltan o están incompletas.
Deficiente	El documento es muy incompleto o no sigue la estructura solicitada.
Claridad y calidad de la documentación (peso: 20 %)	
Excelente	Documento bien organizado, claro y profesional. Uso adecuado de Markdown, tablas, figuras y notación técnica.
Satisfactorio	Documento comprensible pero con problemas menores de organización o presentación.
Regular	Organización deficiente o redacción poco clara.
Deficiente	Documento difícil de leer o mal presentado.

A.2. Presentación funcional

Presentación funcional

Rúbrica aplicable a la demostración funcional de cada proyecto del curso. Evalúa el funcionamiento de cada etapa esperada del sistema por separado, la calidad de la verificación por simulación de la parte descrita en SystemVerilog, y la defensa técnica del diseño ante preguntas del profesor durante la presentación.

Nivel	Descripción del criterio
Subsistema discreto (protoboard) (peso: 15 %)	
Excelente	El circuito discreto (generador pseudoaleatorio, decodificador y codificación serial) funciona de forma completa y estable, generando una nueva posición válida ante cada solicitud y mostrando correctamente el LED del topo activo.
Satisfactorio	El circuito discreto funciona en la mayoría de los casos, con fallas ocasionales en la generación de la posición o en la indicación visual del topo activo.
Regular	El circuito discreto presenta fallas frecuentes en la generación pseudoaleatoria, la decodificación o la indicación visual, aunque se logra demostrar parcialmente su funcionamiento.
Deficiente	El circuito discreto no funciona o no se logra demostrar ninguna parte de su funcionamiento.
Comunicación serial (UART) (peso: 15 %)	
Excelente	La comunicación UART entre el circuito discreto y la FPGA funciona de forma confiable, con el formato de trama especificado (8N1) y sin pérdida ni corrupción de datos durante la demostración.

Nivel	Descripción del criterio
Satisfactorio	La comunicación funciona en la mayoría de los casos, con errores ocasionales de sincronización o pérdida esporádica de tramas que no impiden continuar la demostración.
Regular	La comunicación presenta errores frecuentes que afectan visiblemente el funcionamiento del juego, aunque se logra demostrar que existe transmisión de datos entre ambos subsistemas.
Deficiente	No existe comunicación funcional entre el circuito discreto y la FPGA, o los datos recibidos no corresponden a la posición generada.
Manejo de entradas/salidas en FPGA (peso: 15 %)	
Excelente	Los botones de golpe, el LED de estado y los displays de 7 segmentos funcionan correctamente: las entradas están correctamente sincronizadas y libres de rebotes, y las salidas reflejan en todo momento el estado real del juego.
Satisfactorio	Las entradas y salidas funcionan correctamente en la mayoría de los casos, con fallas ocasionales de rebote, sincronización o actualización de displays.
Regular	Existen fallas evidentes en el manejo de entradas o salidas — rebotes no filtrados, displays incorrectos, LED de estado que no refleja la situación real — que afectan la jugabilidad.
Deficiente	Las entradas o salidas no funcionan de forma reconocible, o el sistema no responde a las pulsaciones del jugador.
Control del juego (FSM y dificultad) (peso: 15 %)	
Excelente	La máquina de estados de control implementa correctamente los turnos, la apertura y cierre de la ventana de tiempo, la reducción progresiva de dicha ventana conforme a lo especificado, y el manejo de vidas y reinicio automático.
Satisfactorio	La lógica de control funciona correctamente en la mayoría de los casos, con desviaciones menores respecto a los parámetros de dificultad o de vidas especificados.
Regular	La lógica de control presenta fallas evidentes en el manejo de turnos, dificultad progresiva o vidas, aunque el juego es parcialmente jugable.
Deficiente	La lógica de control no funciona de forma reconocible, o el juego no sigue las reglas especificadas.
Simulación autoverificable (FPGA/Verilog) (peso: 15 %)	
Excelente	Presenta simulaciones autoverificables (<i>testbenches</i> con verificación automática de resultados) para los bloques principales descritos en SystemVerilog, con reporte claro de aciertos/fallos y cobertura de los casos relevantes, incluyendo condiciones de borde.
Satisfactorio	Presenta simulaciones autoverificables para la mayoría de los bloques, aunque con cobertura limitada de casos de borde o verificación parcialmente manual.

Nivel	Descripción del criterio
Regular	Las simulaciones presentadas requieren inspección manual de formas de onda para determinar su correctud; cobertura escasa de los bloques del diseño.
Deficiente	No presenta simulaciones autoverificables, o las presentadas no permiten verificar el comportamiento correcto del diseño.
Defensa y comprobación de dominio técnico (peso: 25 %)	
Excelente	Responde con precisión y seguridad a las preguntas técnicas sobre el diseño, justificando decisiones de arquitectura e implementación, e identificando con claridad el funcionamiento de cada bloque ante modificaciones o pruebas solicitadas por el profesor durante la presentación.
Satisfactorio	Responde correctamente a la mayoría de las preguntas técnicas, aunque con dudas puntuales o explicaciones incompletas sobre algunos bloques del diseño.
Regular	Responde de forma superficial o memorística, con dificultad para justificar decisiones de diseño o para explicar el funcionamiento ante cambios solicitados.
Deficiente	No logra explicar ni justificar el funcionamiento del diseño, o las respuestas evidencian que no comprende su propia implementación.

A.3. Informe técnico

Documentación técnica (informe)

Rúbrica aplicable al informe técnico final entregado en cada proyecto del curso. Evalúa la fundamentación teórica, la presentación y el análisis de los resultados, las conclusiones obtenidas y la calidad general del documento.

Formato de entrega: el informe técnico debe entregarse como un archivo Markdown (.md) enriquecido, ubicado en la carpeta docs/informe/ del repositorio del proyecto. No se aceptan formatos alternativos, como un PDF exportado desde un procesador de texto externo, como entrega oficial.

Nivel	Descripción del criterio
Fundamentación teórica (peso: 20 %)	
Excelente	Presenta los conceptos teóricos relevantes con claridad y correcta relación con el laboratorio/proyecto. Integra adecuadamente el marco teórico con el trabajo realizado.
Satisfactorio	Presenta la mayoría de los conceptos necesarios, aunque con poca profundidad o conexión limitada con el laboratorio/proyecto.
Regular	Marco teórico superficial o incompleto.
Deficiente	No presenta fundamentación teórica o es incorrecta.

Nivel	Descripción del criterio
Presentación de resultados (peso: 30 %)	
Excelente	Resultados completos y organizados (tablas, gráficas, simulaciones, mediciones). Claramente etiquetados y bien documentados.
Satisfactorio	Resultados mayormente completos, aunque con organización o explicación limitada.
Regular	Resultados incompletos o presentados de forma poco clara.
Deficiente	No presenta resultados relevantes o son incorrectos.
Análisis e interpretación de resultados (peso: 25 %)	
Excelente	Analiza los resultados de forma crítica, compara valores teóricos, simulados y experimentales cuando aplica, e identifica causas de diferencias o errores.
Satisfactorio	Presenta análisis general de los resultados, pero con poca profundidad o discusión limitada.
Regular	Análisis superficial, descriptivo o con escasa interpretación técnica.
Deficiente	No analiza los resultados o las conclusiones no se basan en los datos.
Conclusiones y aprendizaje obtenido (peso: 15 %)	
Excelente	Conclusiones claras, bien fundamentadas y directamente relacionadas con los resultados y objetivos del laboratorio/proyecto. Incluye reflexiones o lecciones aprendidas.
Satisfactorio	Conclusiones correctas pero generales o poco conectadas con los resultados.
Regular	Conclusiones superficiales o parcialmente relacionadas con el trabajo realizado.
Deficiente	No presenta conclusiones o son incorrectas.
Calidad y organización del documento (peso: 10 %)	
Excelente	Documento claro, bien estructurado y profesional. Uso adecuado de Markdown, figuras, tablas, ecuaciones y referencias.
Satisfactorio	Organización adecuada con pequeños problemas de claridad o formato.
Regular	Documento desorganizado o difícil de seguir.
Deficiente	Documento mal estructurado o incompleto.

A.4. Repositorio

Gestión de repositorio

Rúbrica aplicable al repositorio Git de cada proyecto del curso. Evalúa el historial de commits, el uso de ramas de desarrollo, la gestión de issues, la revisión colaborativa de cambios y la estructura del README.

Nivel	Descripción del criterio
Historial de commits y calidad de código (peso: 20 %)	
Excelente	Commits frecuentes, claros y descriptivos, reflejan avances reales del proyecto y buena trazabilidad. Código documentado y organizado correctamente.
Satisfactorio	Commits suficientes pero con mensajes genéricos o frecuencia irregular. Código sin comentar o desordenado.
Deficiente	Pocos commits, sin descripción clara o sin reflejar el desarrollo. No tiene código real.
Uso de ramas de desarrollo (peso: 20 %)	
Excelente	Uso sistemático de ramas por funcionalidad o integrante, siguiendo un flujo profesional (Git Flow o similar).
Satisfactorio	Se usan ramas ocasionalmente o sin convención clara.
Deficiente	Todo el trabajo se hace en una sola rama sin control de versiones adecuado.
Gestión de issues y seguimiento (peso: 20 %)	
Excelente	Issues bien definidos y organizados, usados para planificar tareas, reportar bugs y documentar avances.
Satisfactorio	Uso limitado de issues o sin seguimiento completo.
Deficiente	No se usan issues o no hay evidencia de planeamiento.
Revisión colaborativa (pull/merge requests) (peso: 20 %)	
Excelente	Uso activo de pull o merge requests con revisiones cruzadas entre miembros antes de integrar cambios.
Satisfactorio	Uso ocasional o sin revisión efectiva.
Deficiente	No se utilizan revisiones ni solicitudes de integración.
Estructura del README (peso: 20 %)	
Excelente	README completo y organizado: incluye propósito, dependencias, instalación, compilación y ejecución.
Satisfactorio	README con información parcial o poco detallada.
Deficiente	README ausente o sin instrucciones útiles.